Rajarata University of Sri Lanka
Department of Computing

ICT1402
PRINCIPLES OF PROGRAM DESIGN & PROGRAMMING

Lecture 04

Pseudocodes

K.A.S.H. Kulathilake
Ph.D., M. Phil., MCS, B.Sc. (Hons) IT, SEDA(UK)

Objectives

- At the end of this lecture students should be able to;
  - Define and apply the rules of pseudocode.
  - Convert a flowchart to pseudocode and vise-versa.
  - Describe the advantages and disadvantages of pseudocode.
  - Describe the advantages and disadvantages of flowcharting.
  - Design programs in pseudocode that include control structures and multiple modules.

Introduction

- Another formal program design tool.
- It was developed along with the structured programming.
- Pseudocode works very well with the rules of structured design and programming.

Rules for Pseudocode

- Write only one statement per line.
- Capitalize initial key word.
- Indent to show hierarchy.
- End multiline structures.
- Keep statements language independent.

Rules for Pseudocode (Cont...)

- **Write only one statement per line**
  - In the task list, we have major tasks, sub tasks and sub- sub tasks.
  - We now convert each of those detail elements into one line in our pseudocode.
  - Each pseudocode line will represent one "action" in our design or,will mark the boundaries of an action.

Rules for Pseudocode (Cont…)

- Payroll example
  - We will assume our input includes name, the hours worked, and the hourly rate and that we are to write out the gross pay as well as the gross pay minus a deduction of $15 for dues and uniforms.
    - Task list

Read name, hourly rate, hours worked
Calculate gross pay and net pay
Write name, gross pay, net pay

Rules for Pseudocode (Cont…)

- Pseudocode

```
READ name, hourly rate, hours worked
Gross pay = hourly rate times hours worked
Net pay = Gross pay minus 15
WRITE name, Gross pay, Net pay
```

- Notice that each statement is a single action and is written on a separate line.

Rules for Pseudocode (Cont...)

- Capitalize initial keyword
  - READ and WRITE are written in all capitals.
  - They are "initial keywords" because they begin the statement, and they are command words that give special meaning to the operation.
  - In pseudocode, we will use a limited set of keywords, primarily READ,WRITE, IF, WHILE, and UNTIL.

```
READ name, hourly rate, hours worked
Gross pay = hourly rate times hours worked
Net pay = Gross pay minus 15
WRITE name, Gross pay, Net pay
```

Rules for Pseudocode (Cont...)

- Indent to show hierarchy
  - Indentation is one of the ways we use to show the "boundaries" of the basic structures such as conditions and loops.
  - As long as we have only sequence structures, we will not have indentation because we do not have any questions of hierarchy.

Rules for Pseudocode (Cont…)

```
READ name, hourly rate, hours worked
Gross pay = hourly rate times hours worked
Net pay = Gross pay minus 15
WRITE name, Gross pay, Net pay
```

We have used indentation to show which statements are within IF.

```
READ name, hourly rate, hours worked
Gross pay = hourly rate times hours worked
Net pay = Gross pay minus 15
IF hours worked is greater than 40
    Net pay = Net pay minus 10
ENDIF
WRITE name, Gross pay, Net pay
```

Rules for Pseudocode (Cont…)

- End multi-line structures
  - Including end multi-line structure makes sure the reader recognizes that we have finished with that structure and are now going on to a new statement.
  - Apply at the end of selection and loop blocks.

```
READ name, hourly rate, hours worked
Gross pay = hourly rate times hours worked
Net pay = Gross pay minus 15
IF hours worked is greater than 40
    Net pay = Net pay minus 10
ENDIF
WRITE name, Gross pay, Net pay
```

Rules for Pseudocode (Cont…)

- Keep statements language independent
  - You should not let your pseudocode turn into the code that you will eventually write in.

As we know, there is not standardization of the form to use in pseudocode. We will add to our "rules" by developing a format for our design structures.

Advantages and Disadvantages of Pseudocode

- Advantages
  - It can be done easily on a word processor, and therefore
  - It is easily modified.
  - It implements structured design elements well.

- Disadvantages
  - It is not visual: We do not get a picture of the design.
  - There is no universal standard on the style or format, so one company's pseudocode might look very different form another's.

Advantages and Disadvantages of Flow Charting

- Advantages
  - It is standardized: all agree on acceptable symbols.
  - It is visual.

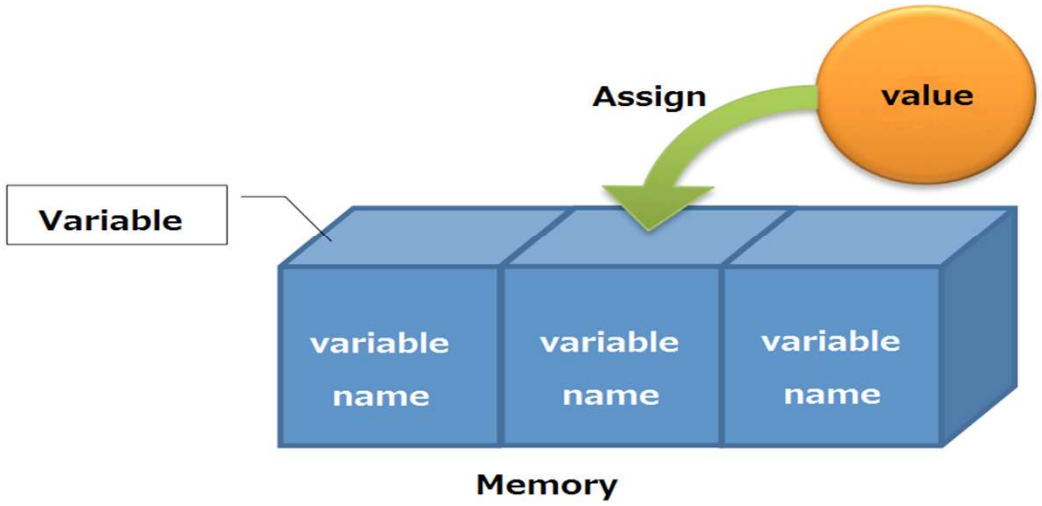
- Disadvantage
  - It is difficult to modify: a simple change may mean redoing the entire flowchart.
  - To do it on a computer, you need a special flowchart package.

Storing and Accessing Data

- When we say READ name, hours worked, hourly rate, we are saying that we want the computer to go out to some external input device and retrive three values.
- These values will be stored in the computer's memory,
- Each memory location has an address, and we could use that memory address as the way we refer to the value when giving instructions to the computer.
  - We could say, READ a numeric value, store it at address 61253.
  - We could tell the computer to multiple the values at addresses 61253 and 67952 and store the results of that multiplication at address 71234.
- But as you can see, it would get very cumbersome very soon.

Storing and Accessing Data (Cont…)

- The computer refers to the memory locations by address, but our programming languages allow us to identify a name for the memory locations.
- The value that us stored in memory is called a variable, and the name we use to refer to it is the variable name.
- So when we say READ hours worked, we are really saying, "READ a value and store it in a memory address that I will refer to as hours worked."

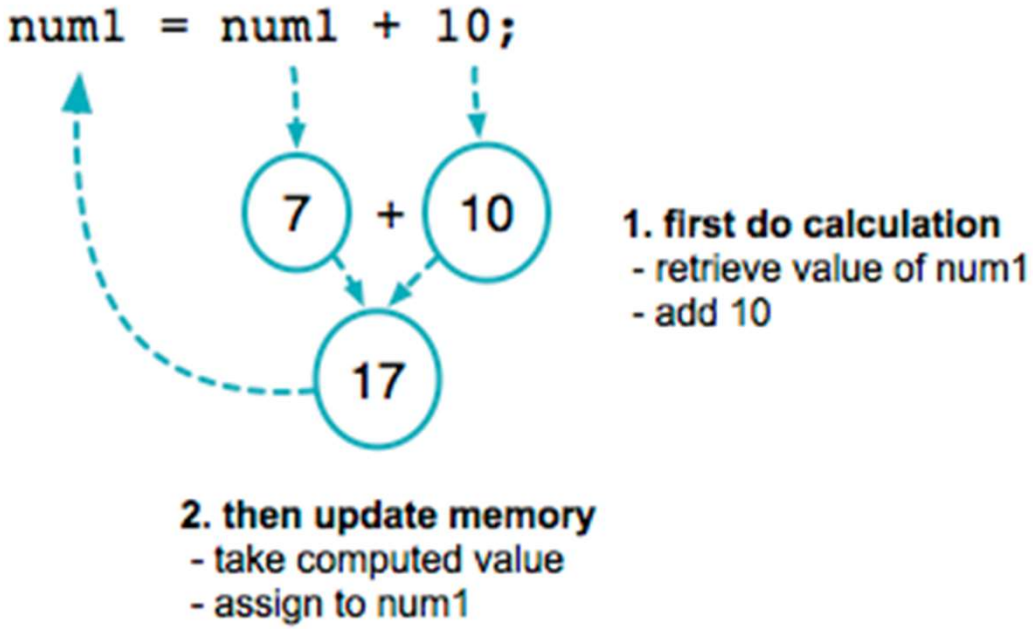


Storing and Accessing Data (Cont…)

- We have to be consistent with our variable names so that there is no confusion when it is time to convert the design to program code.
- Do not put the space between the words of the variable names.
- It is better to use lowercase and capitals consistently when declaring variables.

Storing and Accessing Data (Cont…)

- Assignment are also possible in calculations.
- Assignment always go from the right side of the = sign to the left side.



Calculation Symbols

ARITHMETIC SYMBOLS FOR CALCULATIONS

/	Division	+	Addition
*	Multiplication	-	Subtraction
^ or **	Exponentiation		

Design Structures in Pseudocode

- Sequence
  - The sequence is just an ordered list of statements, we implement it in pseudocode by giving an ordered list of pseudocode statements.
- Selection

```
IF hoursWorked is greater than 40
    regularPay = hourlyRate times 40
    overtimeHours = hoursWorked minus 40
    overtimePay = overtimeHours * hourlyRate * 2
    grossPay = regularPay plus overtimePay
ELSE
    grossPay = hoursWorked times hourlyRate
ENDIF
```

Design Structures in Pseudocode (Cont…)

- The indent lines between the IF and the ELSE from the "true" section- the statements to be performed if the condition is true.
- The indented lines between the ELSE and the ENDIF form the "false" section - the statements to be performed if the condition is false.
- representing one-sided selection structures when we only had statements to perform if the condition was true.

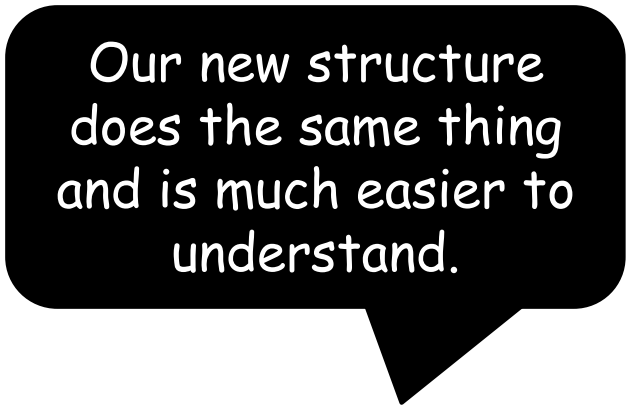
```
IF hoursWorked is greater than 40
    netPay = netPay minus 10
ENDIF
```

Design Structures in Pseudocode (Cont…)

- Special scenario with the condition section of a selection.
- Assume that we want to write num3 if it is not grater than num4

```
IF num3 greater than num4

ELSE
    WRITE num3
ENDIF
```




```
IF num3 less than or equal to num4
    WRITE num3
ENDIF
```

Design Structures in Pseudocode (Cont…)

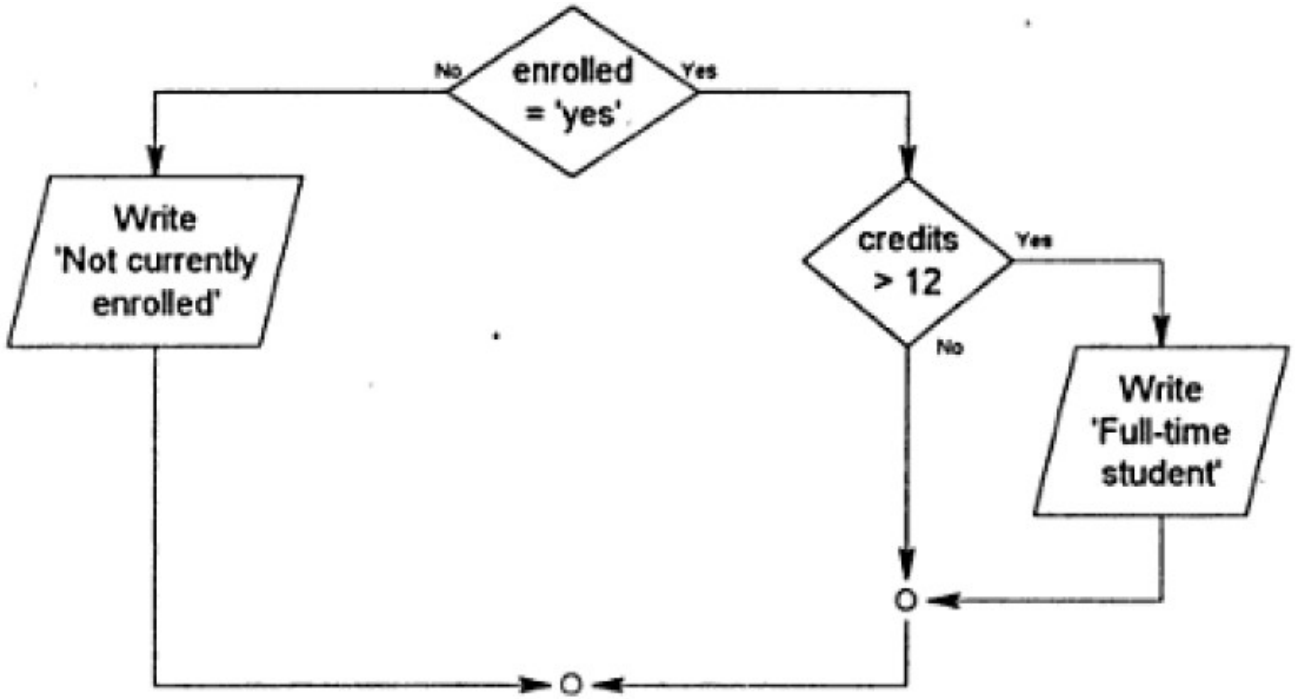
- Relational conditional symbols are possible in section construct

RELATIONAL CONDITION SYMBOLS

<	Less than	<=	Less than or equal to
>	Greater than	>=	Greater than or equal to
=	Equal to	<>	Not equal to

Design Structures in Pseudocode (Cont…)

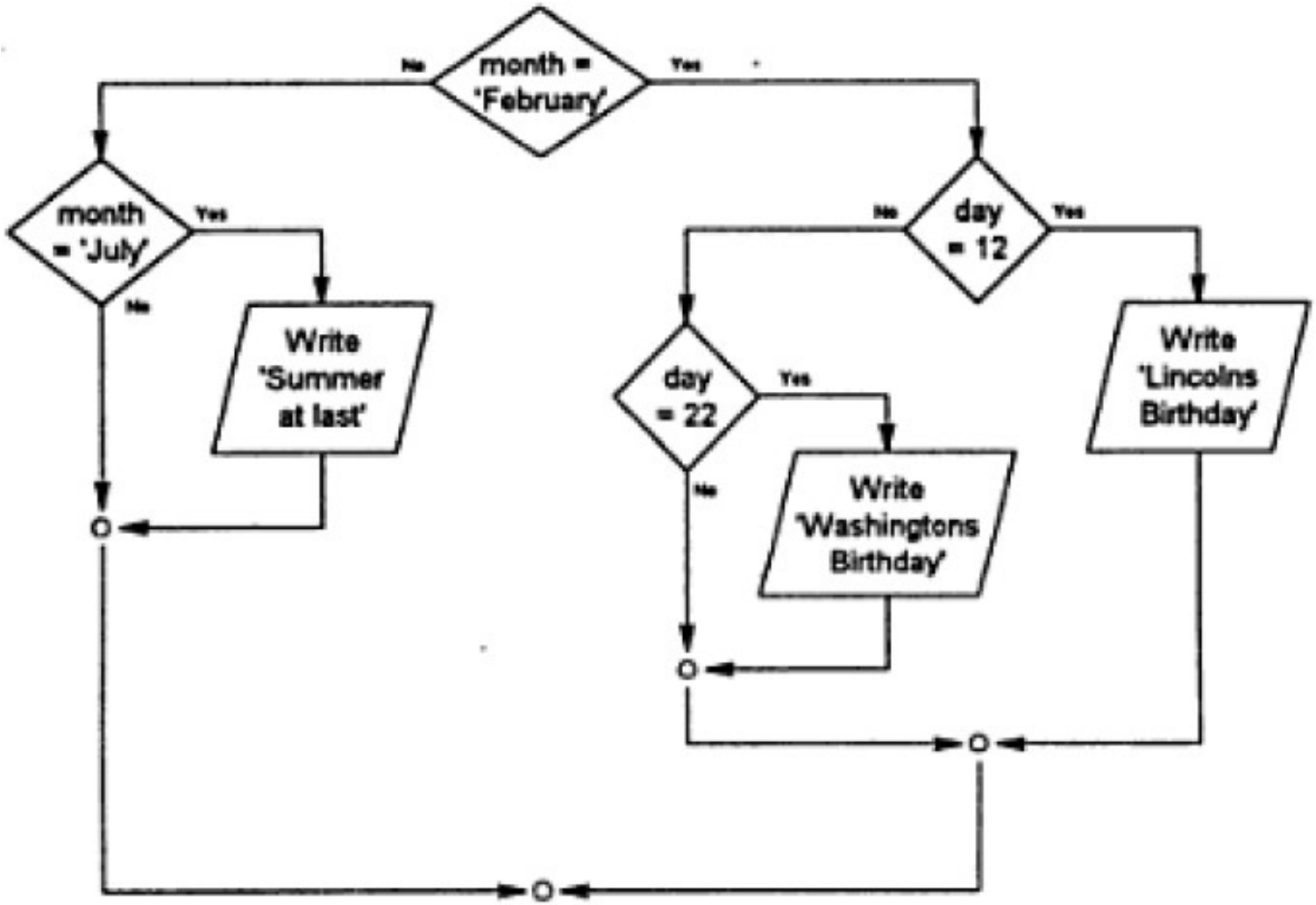
- Nested IF




```
IF enrolled = 'yes'
    IF credits > 12
        WRITE 'Full-time student'
    ENDIF
ELSE
    WRITE 'Not currently enrolled'
ENDIF
```

Design Structures in Pseudocode (Cont…)

- Try this



Design Structures in Pseudocode (Cont…)

- Answer

```
IF month = 'February'
    IF day = 12
        WRITE 'Lincolns Birthday'
    ELSE
        IF day = 22
            WRITE 'Washingtons Birthday'
        ENDIF
    ENDIF
ELSE
    IF month = 'July'
        WRITE 'Summer at last'
    ENDIF
ENDIF
```

Design Structures in Pseudocode (Cont…)

- Loops / Iterations
  - In pseudocode, we use the keyword WHILE to introduce a while loop and the ENDWHILE to mark the end of the loop.

```
count = 0
WHILE count < 10
    READ name
    WRITE name
    ADD 1 to count
ENDWHILE
WRITE 'The End'
```

Design Structures in Pseudocode (Cont…)

- Calling a module

```
Mainline
count = 0
WHILE count < 10
    DO Process
ENDWHILE
WRITE 'The End'

Process
READ name
WRITE name
ADD 1 to count
```

We pass control to the module by using the keyword DO, meaning "Go do the module called Process and when you finish that module, return to the statement that follows."

Flowcharts have terminal symbols at each end of a module.
Pseudocode usually starts each module with a name but does nothing to indicate the end of a module.

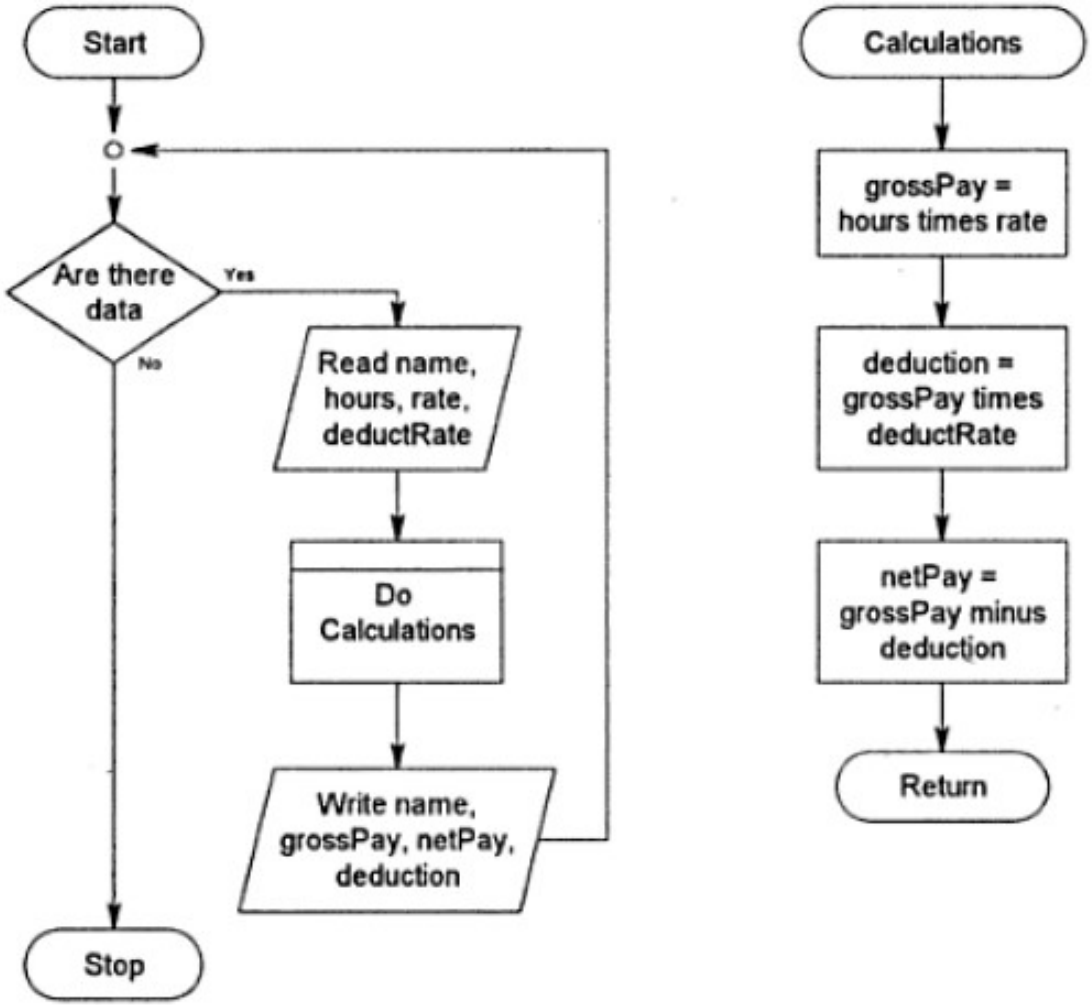
Design Structures in Pseudocode (Cont…)

- Until loop

```
Mainline
Count = 0
REPEAT
    READ name
    WRITE name
    ADD 1 to count
UNTIL count > = 10
WRITE 'The End'
```

UNTIL loop tests the condition at the END of the loop, and we exit the loop when the condition is true. With an UNTIL loop we are using the keyword REPEAT to mark the beginning of the loop body and UNTIL to mark the end of the loop body.

Example


Start
Are there data
Yes
No
Read name, hours, rate, deductRate
Do Calculations
Write name, grossPay, netPay, deduction
Stop
Calculations
grossPay = hours times rate
deduction = grossPay times deductRate
netPay = grossPay minus deduction
Return

Answer

```
Mainline
totalPay = 0
WHILE there are data
    DO Process
ENDWHILE
WRITE totalPay

Process
READ name, payRate, hoursWorked, deductionRate
grossPay = payRate * hoursWorked
deductions = grossPay * deductionRate
netPay = grossPay - deductions
ADD grossPay to totalPay
WRITE name, deductions, netPay, grossPay
```

Algorithm

- An algorithm is a procedure or formula for solving a problem, based on conducting a sequence of specified actions.
- A computer program can be viewed as an elaborate algorithm.
- In mathematics and computer science, an algorithm usually means a small procedure that solves a recurrent problem.

Difference between Algorithm and Pseudocode

```
Algorithm: Compute the area of rectangle
BEGIN
        GET values for length and width
        SET area to 0
        calculate area by multiplying length and width
        DISPLAY area
END
```

```
Pseudocode: Compute the area of rectangle

READ length, width
SET area = 0
area = length * width
print area
```

All Together

- An **algorithm** is a well-defined procedure that allows a computer to solve a problem.
- A **Pseudocode** is a simple way of writing programming code in English. Pseudocode is not an actual programming language. It uses short phrases to write code for programs before you actually create it in a specific language.
- A **program** is a sequence of instructions in a particular programming language, written to perform a specified task on a computer.

Objectives - Revisit

- Define and apply the rules of pseudocode.
- Convert a flowchart to pseudocode and vise-versa.
- Describe the advantages and disadvantages of pseudocode.
- Describe the advantages and disadvantages of flowcharting.
- Design programs in pseudocode that include control structures and multiple modules.

References

- Chapter 03 of Computer Program Design, Elizabeth A. Dickson.

Q & A

End of Program Design Module

Let's start Programming Module in the next session